

Spring Boot 4 и Spring Framework 7: Ключевые фишки и изменения

Простой

10 мин

44К

Блог компании Spring АйОПрограммирование*Java*Kotlin*

Обзор

Перевод

Автор оригинала: Loiane Groner

Уже в ноябре 2025 выходит Spring Boot 4 — масштабный релиз, построенный на фундаменте Spring Framework 7.0.0 и Jakarta EE 11. Это не просто очередное обновление, а важный этап в развитии всей Java-платформы: новые возможности для REST и безопасности, улучшенная работа с Kotlin и GraalVM, мощная поддержка облаков и нативных образов, а также инструменты для создания отказоустойчивых приложений.

□ Эта статья — больше, чем просто перевод: в ней собрано большое кол-во комментариев от экспертов *Spring АйО*.

Spring Boot 4 выйдет в ноябре 2025 года (milestone версии уже доступны) и основан на совершенно новой версии Spring Framework 7.0.0. Если вы разрабатываете современные Java-приложения, это обновление предлагает множество новых фишек, с которыми стоит ознакомиться.

После масштабной миграции в Spring Boot 3.x на Jakarta EE 10, Spring Boot 4 продолжает курс на модернизацию и основывается на Jakarta EE 11. Spring Boot 4 также предлагает поддержку современных версий JDK и практик облачной разработки. Это важный этап, который задаёт вектор развития Java на ближайшее десятилетие.

В этой статье мы рассмотрим ключевые изменения и их значение для ваших проектов. Обновления платформы Spring Boot 4

1. Поддержка Java и JVM

- Минимальная версия JDK: 17 (LTS и широко используется).
- Рекомендуемая версия JDK: 25 (релиз ожидается в сентябре 2025 года).
- Полная совместимость с GraalVM 24 для сборки нативных образов.

2. Совместимость с Jakarta EE 11

- Servlet 6.1 для современных веб-API.
- JPA 3.2 + Hibernate ORM 7.0 для более эффективной работы с данными.
- Bean Validation 3.1 с улучшенными ограничениями и поддержкой Kotlin/records.

3. Поддержка Kotlin 2.2

- Поддержка новых возможностей языка Kotlin 2.2.
- Улучшенная работа с корутинами.
- Более тесная интеграция со сборкой Gradle.

4. Облачные технологии и контейнеризация

- Улучшенные Buildpacks с ускоренным созданием слоёв для образа.
- Более эффективная сборка Docker-образов.
- Интеграция Micrometer 2.x и OpenTelemetry.

5. Продуктивность и удобство разработки

- Обновления Spring Boot CLI (управление зависимостями, генерация шаблонов для тестов).
- Новые эндпоинты в Actuator.
- Более простое управление свойствами и профилями.

6. Безопасность

- Основано на Spring Security 7.
- Улучшенная интеграция с OAuth 2.2 и OIDC.
- Улучшенные криптографические настройки по умолчанию.

7. Подготовка к будущему: Spring AI и нативные API

- Начальная поддержка интеграции со Spring AI.
- Улучшенный API для “подсказок” GraalVM при сборке нативных образов.

Комментарий от эксперта Spring АйО, Михаила Поливахи

Новые возможности Spring Framework 7

1. Элегантное версионирование API

Spring Framework 7 теперь поддерживает версионирование API напрямую в аннотации `@RequestMapping`, что упрощает сопровождение нескольких версий REST-эндпоинтов и обеспечивает обратную совместимость.

```
@RestController
```

```
@RequestMapping("/api/products")
```

```
public class ProductController {
```

```
    @RequestMapping(value = "/search", version = "1")
```

```
    public List<ProductV1> searchProductsV1(@RequestParam String query) { ... }
```

```
    @RequestMapping(value = "/search", version = "2")
```

```
public ProductSearchResponseV2 searchProductsV2(@RequestParam String query,  
                                                @RequestParam(defaultValue = "10") int limit) { ... }  
}
```

[Объяснить с](#)

Вы можете указать нужную версию с помощью заголовка `Version` в HTTP-запросе.

Контроллер автоматически направит запрос в соответствующий метод в зависимости от указанной версии.

Пример вызова версии 1 API (возвращает простой список продуктов):

```
GET /api/products/search?query=laptop
```

Version: 1

[Объяснить с](#)

Пример вызова версии 2 API (возвращает расширенный поиск с метаданными):

```
GET /api/products/search?query=laptop&limit=20
```

Version: 2

[Объяснить с](#)

Комментарий от эксперта Spring АйО, Михаила Поливахи

2. Безопасность при работе с null с помощью JSpecify

Spring Framework 7 перешёл на использование аннотаций JSpecify, что повышает безопасность при работе с null и улучшает интеграцию с Kotlin. JSpecify предоставляет более точные контракты на null-значения для параметров методов, возвращаемых значений и полей, помогая разработчикам обнаруживать потенциальные ошибки, связанные с null, на этапе компиляции.

Комментарий от эксперта Spring АйО, Михаила Поливахи

Пример использования аннотаций JSpecify в компоненте Spring:

```
import org.jspecify.annotations.Nullable;  
import org.jspecify.annotations.NonNull;  
  
@Component  
public class UserService {  
    public @NonNull String getUserName(@Nullable String userId) {
```

```

    if (userId == null) {
        return "Unknown";
    }
    // fetch user name logic
    return "Alice";
}
}

```

[Объяснить с](#)

В этом примере параметр `userId` может быть `null`, но метод гарантирует возвращаемое значение, отличное от `null`. Это помогает предотвратить ошибки типа `NullPointerException` и повышает надёжность кода, особенно при интеграции с Kotlin или другими языками с безопасной работой с `null`.

3. Программная регистрация бинов с помощью BeanRegistrar

Spring Framework 7 представляет контракт `BeanRegistrar`, позволяющий разработчикам программно регистрировать бины с большей гибкостью. Это особенно полезно в сложных сценариях, когда необходимо динамически регистрировать несколько бинов — за пределами возможностей, предоставляемых аннотацией `@Bean`.

Комментарий от эксперта Spring АйО

```

@Configuration(proxyBeanMethods = false)
public class MyBeanConfig implements BeanRegistrar {
    @Override
    public void registerBeans(BeanDefinitionRegistry registry) {
        registry.registerBeanDefinition("myBean", new RootBeanDefinition(MyBean.class));
    }
}

```

[Объяснить с](#)

Вы можете регистрировать бины на основе условий выполнения или внешней конфигурации, делая приложение более динамичным и модульным.

4. Улучшенная поддержка Optional в выражениях SpEL

SpEL (Spring Expression Language) теперь предлагает расширенную поддержку типов `Optional`, включая `null-safe` вызовы и оператор Элвиса (`?:`). Это упрощает работу с `Optional`-значениями в конфигурациях и определениях бинов.

```
@Value("#{userService.findUser(#userId)?.orElse('Guest')}")
```

```
private String userName;
```

[Объяснить с](#)

Вы можете безопасно обрабатывать отсутствующие значения и избегать `NullPointerException` в конфигурации и связывании бинов.

5. Функционал отказоустойчивости: `@Retryable`, `@ConcurrencyLimit`, `@EnableResilientMethods`

Новые аннотации, такие как `@Retryable` и `@ConcurrencyLimit`, позволяют создавать более устойчивые приложения, реализуя логику повторных попыток и ограничения параллелизма прямо в методах сервисов.

Аннотация `@EnableResilientMethods` активирует эти возможности для ваших бинов.

```
@Service
```

```
@EnableResilientMethods
```

```
public class PaymentService {
```

```
    @Retryable(maxAttempts = 3)
```

```
    public void processPayment() {
```

```
        // logic that may fail and should be retried
```

```
    }
```

```
    @ConcurrencyLimit(maxConcurrentCalls = 5)
```

```
    public void updateBalance() {
```

```
        // logic with concurrency limit
```

```
    }
```

```
}
```

[Объяснить с](#)

Вы можете добавить повторные попытки и контроль параллелизма в бизнес-логику с помощью простых аннотаций, повышая надёжность и масштабируемость приложения.

Комментарий от эксперта Spring АйО, Михаила Поливахи

6. Отдельная конфигурация для HTTP-клиентов с помощью `@ImportHttpServices`
Spring Framework 7 вводит аннотацию `@ImportHttpServices`, упрощающую настройку и группировку HTTP-клиентов. Это оптимизирует конфигурацию приложений, взаимодействующих с несколькими внешними сервисами.

```
@Configuration(proxyBeanMethods = false)
@ImportHttpServices(group = "weather", types = {WeatherClient.class})
public class HttpClientConfig {}
```

[Объяснить с](#)

Вы можете эффективно организовать и настроить несколько HTTP-клиентов, сократив boilerplate код и повысив удобство поддержки.

Комментарий от эксперта Spring АйО, Михаила Поливахи

7. Поддержка потоковой передачи с `InputStream` и `OutputStream` в HTTP-клиентах

HTTP-клиенты теперь поддерживают потоковую передачу тел запросов и ответов с использованием `InputStream` и `OutputStream`. Это особенно полезно для эффективной обработки больших файлов и потоков данных.

```
@PostMapping("/upload")
public void uploadFile(InputStream inputStream) {
    // process large file stream
}
```

[Объяснить с](#)

Вы можете обрабатывать загрузку и выгрузку больших файлов без полной загрузки данных в память, что повышает производительность и масштабируемость.

8. Новый `JmsClient` и улучшения в `JdbcClient`

В Spring Framework 7 представлен новый `JmsClient` — современный API для работы с JMS (Java Message Service), а также усовершенствованный `JdbcClient` для более простой и гибкой работы с базами данных.

```
JmsClient jmsClient = JmsClient.create(connectionFactory);
jmsClient.send("queue", "Hello World");

JdbcClient jdbcClient = JdbcClient.create(dataSource);
List<User> users = jdbcClient.sql("SELECT * FROM users").query(User.class).list();
```

[Объяснить с](#)

Вы получаете более лаконичный и современный API для обмена сообщениями и доступа к данным, что сокращает boilerplate код и повышает продуктивность разработки.

9. Централизованная конфигурация HTTP-конвертеров сообщений

Spring Framework 7 предлагает централизованную настройку HTTP-конвертеров сообщений, что упрощает кастомизацию процессов сериализации и десериализации данных в веб-приложениях.

@Configuration

```
public class WebConfig implements WebMvcConfigurer {  
    @Override  
    public void configureMessageConverters(HttpMessageConverters.ServerBuilder builder) {  
        builder.jsonMessageConverter(new  
JacksonJsonHttpMessageConverter(JsonMapper.builder().build()));  
    }  
}
```

[Объяснить с](#)

Вы можете легко настраивать логику сериализации и десериализации для ваших API в одном месте, повышая согласованность и удобство сопровождения кода.

10. Новый RestTestClient для тестирования REST API

RestTestClient — это новый инструмент для тестирования REST API, предоставляющий лаконичный API и поддержку как реальных серверов, так и мок-окружений. Он упрощает написание и сопровождение интеграционных тестов для ваших эндпойнтов.

```
RestTestClient client = RestTestClient.bindToServer("http://localhost:8080");  
client.get().uri("/api/user").exchange().expectStatus().isOk();
```

[Объяснить с](#)

Вы можете писать лаконичные и читаемые интеграционные тесты для ваших REST-эндпойнтов, повышая масштаб покрытия и надёжность тестирования.

11. Улучшенное сопоставление путей с расширенной поддержкой PathPattern

В Spring MVC улучшен механизм сопоставления путей благодаря расширенной поддержке PathPattern, которая заменяет устаревшие варианты и обеспечивает более мощное и гибкое сопоставление URI-шаблонов в контроллерах.

Комментарий от эксперта Spring АйО, Михаила Поливахи

```
@RequestMapping("/**/pages/{pageName}")  
public String handlePage(@PathVariable String pageName) {
```

```
return pageName;  
}
```

[Объяснить с](#)

Вы можете задавать более гибкие и выразительные шаблоны URI для контроллеров, упрощая маршрутизацию и проектирование эндпоинтов.

12. Удаленный и deprecation API

Spring Boot 4 и Spring Framework 7 исключили ряд устаревших функций и поместили как устаревшие некоторый API, чтобы упростить экосистему и стимулировать использование современных практик разработки.

Removed / Deprecated	Alternative
spring-jcl	Apache Commons Logging
javax.annotation, javax.inject	jakarta.annotation, jakarta.inject
suffixPatternMatch, trailingSlashMatch, favorPathExtension	Explicit media types / URI templates
XML-based Spring MVC config	Java-based WebMvcConfigurer
JUnit 4	JUnit 5
Jackson 2.x	Jackson 3.x

Ниже представлены подробности по каждому удалённому и устаревшему элементу.

Изменения в модулях

Модуль spring-jcl был удалён в пользу использования Apache Commons Logging, что упрощает инфраструктуру логирования. Кроме того, аннотации из пакетов javax.annotation и javax.inject больше не поддерживаются — необходимо перейти на их аналоги из пространства имён Jakarta: jakarta.annotation и jakarta.inject.

Комментарий от команды эксперта Spring АйО Михаила Поливахи

Упрощение сопоставления путей

В Spring MVC были удалены несколько опций маршрутизации путей. Такие функции, как suffixPatternMatch, trailingSlashMatch и favorPathExtension, больше не поддерживаются в аннотированных методах контроллеров и при согласовании форматов контента.

Ранее можно было использовать маршрутизацию на основе суффиксов, например:

```
@RequestMapping(value = "/users.json", produces = "application/json")
public List<User> getUsersJson() { ... }
```

[Объяснить с](#)

Или настроить сопоставление путей по суффиксам в конфигурации на Java следующим образом:

```
@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Override
    public void configurePathMatch(PathMatchConfigurer configurer) {
        configurer.setUseSuffixPatternMatch(true);
    }
}
```

[Объяснить с](#)

Эти опции теперь удалены. Вместо них рекомендуется использовать явные медиа-типы и шаблоны URI, обеспечивающие более чистую и предсказуемую маршрутизацию.

Устаревание конфигурации на XML

Конфигурация Spring MVC с использованием XML теперь считается устаревшей. Если вы всё ещё используете конфигурацию на основе XML, например:

```
<mvc:annotation-driven />
<mvc:resources mapping="/resources/**" location="/public/" />
<mvc:view-controller path="/home" view-name="home" />
```

[Объяснить с](#)

Рекомендуется перейти на конфигурацию на основе Java, например:

```
@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Override
    public void addViewControllers(ViewControllerRegistry registry) {
        registry.addViewController("/home").setViewName("home");
    }
}
```

```
}  
  
}
```

[Объяснить с](#)

Обновления в тестировании и сериализации

Поддержка JUnit 4 и Jackson 2.x объявлена устаревшей. Рекомендуется переход на JUnit 5 и более новые версии Jackson для повышения производительности и использования современных возможностей тестирования.

Чек-лист миграции

- Обновитесь до JDK 17+ (рекомендуется JDK 25).
- Замените необходимые импорты `javax.*` на `jakarta.*`.
- Проверьте настройки маршрутизации в MVC — механизмы сопоставления по суффиксу (`suffixPatternMatch`) и завершающему слэшу (`trailingSlashMatch`) удалены.
- Перейдите с конфигураций на XML к конфигурации на Java.
- Обновите тесты: используйте JUnit 5 и Jackson 3.x.
- Проверьте "подсказки" GraalVM, если используете нативные образы.

Изменения, которые уже можно внедрять: переход на Java 17+, обновление пространств имён пакетов, отказ от XML-конфигураций и переход на JUnit 5 — это поможет упростить миграцию и подготовить проект заранее.

Заключение

Spring Boot 4 и Spring Framework 7 — это значительный шаг вперёд для Java-разработки, предлагающий больше безопасности, гибкости и новых возможностей для современных приложений. Если вы планируете обновление, внимательно изучите изменения и тщательно протестируйте проект перед переходом в продакшн.

Актуальная версия Spring Boot 3.5.x будет поддерживаться как open-source до июня 2026 года, поэтому планируйте миграцию заранее.